

AI ASSISTED CODING

LAB-12:Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Name: D.siddhartha

Roll No:2503A51L04

Batch:25BTCAICSB19

Task Description #1:

(Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.
- Instructions:
 - Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order.
 - Ask AI to include time complexity and space complexity in the function docstring.
 - Verify the generated code with test cases.
- Expected Output:
 - A functional Python script implementing Merge Sort with proper documentation

PROMPT: *Generate a Python program that implements the Merge Sort algorithm create a function `merge_sort(arr)` that sorts a list in ascending order include time complexity and space complexity in the function docstring. Verify the generated code with test cases.*

```

task12-1.py > ...
1  def merge_sort(arr):
2      """
3      Sorts a list in ascending order using the Merge Sort algorithm.
4      Time Complexity: O(n log n) in all cases (best, average, worst).
5      Space Complexity: O(n) due to the use of temporary arrays.
6      Args:
7          arr (list): The list of elements to be sorted.
8      Returns:
9          list: A new sorted list in ascending order.
10     """
11     if len(arr) <= 1:
12         return arr
13     # Find the middle point and divide the array
14     mid = len(arr) // 2
15     left_half = merge_sort(arr[:mid])
16     right_half = merge_sort(arr[mid:])
17     # Merge the sorted halves
18     return merge(left_half, right_half)
19 def merge(left, right):
20     """
21     Merges two sorted lists into one sorted list.
22     Args:
23         left (list): First sorted list.
24         right (list): Second sorted list.
25     Returns:
26         list: Merged sorted list.
27     """
28     merged = []
29     i = j = 0
30     # Compare elements from both lists and append the smaller one
31     while i < len(left) and j < len(right):
32         if left[i] < right[j]:
33             merged.append(left[i])
34             i += 1
35         else:
36             merged.append(right[j])
37             j += 1
38     # Append any remaining elements
39     merged.extend(left[i:])
40     merged.extend(right[j:])
41     return merged
42 # Test cases to verify the implementation
43 if __name__ == "__main__":
44     test_cases = [
45         [],
46         [1],
47         [5, 2, 9, 1, 5, 6],
48         [3, 3, 3],
49         [10, -1, 2, 5, 0],
50         [1, 2, 3, 4, 5],
51         [5, 4, 3, 2, 1]
52     ]
53     for idx, test in enumerate(test_cases):
54         print(f"Test case {idx+1}: Input: {test}")
55         print(f"Sorted: {merge_sort(test)}\n")

```

OUTPUT:

```

Test case 1: Input: []
Sorted: []

Test case 2: Input: [1]
Sorted: [1]

Test case 3: Input: [5, 2, 9, 1, 5, 6]
Sorted: [1, 2, 5, 5, 6, 9]

Test case 4: Input: [3, 3, 3]
Sorted: [3, 3, 3]

Test case 5: Input: [10, -1, 2, 5, 0]
Sorted: [-1, 0, 2, 5, 10]

Test case 6: Input: [1, 2, 3, 4, 5]
Sorted: [1, 2, 3, 4, 5]

Test case 7: Input: [5, 4, 3, 2, 1]
Sorted: [1, 2, 3, 4, 5]

```

Observation: AI generated the `merge_sort(arr)` function using the divide-and-conquer strategy. It included a detailed docstring explaining time complexity ($O(n \log n)$) and space complexity ($O(n)$), which made the algorithm clearer. After running the AI-generated code with test cases, I

observed that the list was sorted correctly. This showed me how AI can not only implement the algorithm but also document and verify it systematically

Task Description #2

(Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.
- Instructions:
 - Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.
 - Include docstrings explaining best, average, and worst-case complexities.
 - Test with various inputs.
- Expected Output:
 - Python code implementing binary search with AI-generated comments and docstrings.

PROMPT: *create a binary search function that finds a target element in a sorted list.create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.Include docstrings explaining best, average, and worst-case complexities.Test with various inputs.*

```

task12-2.py > binary_search
1 def binary_search(arr, target):
2     """
3     Performs binary search to find the index of a target element in a sorted list.
4     Time Complexity:
5         Best case: O(1)      (target is at the middle)
6         Average case: O(log n)
7         Worst case: O(log n)
8     Space Complexity: O(1) (iterative approach)
9     Args:
10        arr (list): A sorted list of elements.
11        target: The element to search for.
12    Returns:
13        int: The index of the target if found, else -1.
14    """
15    left, right = 0, len(arr) - 1
16    while left <= right:
17        mid = (left + right) // 2
18        if arr[mid] == target:
19            return mid
20        elif arr[mid] < target:
21            left = mid + 1
22        else:
23            right = mid - 1
24    return -1
25
26 # Test cases to verify the implementation
27 if __name__ == "__main__":
28     test_cases = [
29         ([], 5),
30         ([1], 1),
31         ([1, 2, 3, 4, 5], 3),
32         ([1, 2, 3, 4, 5], 6),
33         ([10, 20, 30, 40, 50], 10),
34         ([10, 20, 30, 40, 50], 50),
35         ([10, 20, 30, 40, 50], 35),
36         ([2, 4, 6, 8, 10, 12], 8),
37     ]
38     for idx, (arr, target) in enumerate(test_cases):
39         result = binary_search(arr, target)
40         print(f"Test case {idx+1}: Array: {arr}, Target: {target} -> index: {result}")

```

OUTPUT:

```

AHASRA SRU/AI assisted/task12-2.py"
Test case 1: Array: [], Target: 5 -> index: -1
Test case 2: Array: [1], Target: 1 -> index: 0
Test case 3: Array: [1, 2, 3, 4, 5], Target: 3 -> index: 2
Test case 4: Array: [1, 2, 3, 4, 5], Target: 6 -> index: -1
Test case 5: Array: [10, 20, 30, 40, 50], Target: 10 -> index: 0
Test case 6: Array: [10, 20, 30, 40, 50], Target: 50 -> index: 4
Test case 7: Array: [10, 20, 30, 40, 50], Target: 35 -> index: -1
Test case 8: Array: [2, 4, 6, 8, 10, 12], Target: 8 -> index: 3
PS C:\Users\DELL\OneDrive\SAHASRA SRU\AI assisted> 

```

Observation: AI implemented the `binary_search(arr, target)` function by repeatedly dividing the sorted list into halves. The function returned the correct index if the element was found, and -1 otherwise. The AI also explained best, average, and worst-case complexities directly in the docstring, which made it easier to connect theory with practice. Testing with multiple inputs confirmed the accuracy, and I learned how AI-generated code can be both optimized and self-explanatory.

Task Description #3

(Real-Time Application – Inventory Management System)

- Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

- Task:

- Use AI to suggest the most efficient search and sort algorithms for this use case.
- Implement the recommended algorithms in Python.
- Justify the choice based on dataset size, update frequency, and performance requirements

Expected Output:

- A table mapping operation → recommended algorithm → justification.
- Working Python functions for searching and sorting the inventory

PROMPT: *Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:*

1. Quickly search for a product by ID or name.

2. Sort products by price or quantity for stock analysis.

suggest the most efficient search and sort algorithms for this use case.

o Implement the recommended algorithms in Python.

o Justify the choice based on dataset size, update frequency, and performance requirements

```

TASK12-3.PY > sort_by_quantity
1  # --- Algorithm Justification ---
2  # For searching by Product ID or Name, we use Python dictionaries (hash tables).
3  # - Dictionary lookup is O(1) on average, making searches extremely fast even for thousands of products.
4  # - This is ideal for inventory systems where staff need instant access to product details.
5  # For sorting by price or quantity, we use Python's built-in sorted() function (Timsort).
6  # - Timsort has O(n log n) time complexity and is highly efficient for real-world data.
7  # - Sorting is performed on-demand for analysis, so it does not impact update speed.
8  # These choices are optimal for:
9  # - Dataset size: Thousands of products fit easily in memory.
10 # - Update frequency: Dictionaries allow fast updates and lookups.
11 # - Performance: Staff get instant search and analysis results.
12 class Product:
13     def __init__(self, product_id, name, price, quantity):
14         self.product_id = product_id
15         self.name = name
16         self.price = price
17         self.quantity = quantity
18     def __repr__(self):
19         return f"{self.name} (ID: {self.product_id}, Price: {self.price}, Qty: {self.quantity})"
20 # Sample Inventory
21 inventory = [
22     Product(101, "Apple", 30, 50),
23     Product(102, "Banana", 10, 100),
24     Product(103, "Orange", 20, 80),
25     Product(104, "Milk", 50, 30),
26     Product(105, "Bread", 25, 60),
27 ]
28 # Build a dictionary for fast ID lookup (O(1) average-case)
29 product_by_id = {product.product_id: product for product in inventory}
30 # Build a dictionary for fast name lookup (O(1) average-case, case-insensitive)
31 product_by_name = {product.name.lower(): product for product in inventory}
32
33 def search_by_id(product_id):
34     """O(1) average-case lookup using dictionary."""
35     return product_by_id.get(product_id, None)
36
37 def search_by_name(name):
38     """O(1) average-case lookup using dictionary."""
39     return product_by_name.get(name.lower(), None)
40
41 def sort_by_price(products, reverse=False):
42     """O(n log n) using Timsort."""
43     return sorted(products, key=lambda p: p.price, reverse=reverse)
44
45 def sort_by_quantity(products, reverse=False):
46     """O(n log n) using Timsort."""
47     return sorted(products, key=lambda p: p.quantity, reverse=reverse)
48
49 # --- User Input for Interaction ---
50 if __name__ == "__main__":
51     while True:
52         print("\nOptions:")
53         print("1. Search by Product ID")
54         print("2. Search by Product Name")
55         print("3. Show Products Sorted by Price (Low to High)")
56         print("4. Show Products Sorted by Quantity (High to Low)")
57         print("5. Exit")
58         choice = input("Enter your choice (1-5): ").strip()
59         if choice == "1":
60             pid = input("Enter Product ID: ").strip()
61             if pid.isdigit():
62                 result = search_by_id(int(pid))
63                 print("Result:", result if result else "Product not found.")
64             else:
65                 print("Invalid Product ID.")
66         elif choice == "2":
67             name = input("Enter Product Name: ").strip()
68             result = search_by_name(name)
69             print("Result:", result if result else "Product not found.")
70         elif choice == "3":
71             print("\nProducts sorted by Price (Low to High):")
72             for p in sort_by_price(inventory):
73                 print(p)
74         elif choice == "4":
75             print("\nProducts sorted by Quantity (High to Low):")
76             for p in sort_by_quantity(inventory, reverse=True):
77                 print(p)
78         elif choice == "5":
79             print("Exiting.")
80             break
81         else:
82             print("Invalid choice. Please try again.")

```

OUTPUT:

```
Options:
1. Search by Product ID
2. Search by Product Name
3. Show Products Sorted by Price (Low to High)
4. Show Products Sorted by Quantity (High to Low)
5. Exit
Enter your choice (1-5): 5
Enter your choice (1-5): 1
Enter Product ID: 103
Result: Orange (ID: 103, Price: 20, Qty: 80)

Options:
1. Search by Product ID
2. Search by Product Name
3. Show Products Sorted by Price (Low to High)
4. Show Products Sorted by Quantity (High to Low)
5. Exit
Enter your choice (1-5): 5
Result: Orange (ID: 103, Price: 20, Qty: 80)

Options:
1. Search by Product ID
2. Search by Product Name
3. Show Products Sorted by Price (Low to High)
4. Show Products Sorted by Quantity (High to Low)
5. Exit
Enter your choice (1-5): 5
Options:
1. Search by Product ID
2. Search by Product Name
3. Show Products Sorted by Price (Low to High)
4. Show Products Sorted by Quantity (High to Low)
5. Exit
Enter your choice (1-5): 5
2. Search by Product Name
3. Show Products Sorted by Price (Low to High)
4. Show Products Sorted by Quantity (High to Low)
5. Exit
Enter your choice (1-5): 5
3. Show Products Sorted by Price (Low to High)
4. Show Products Sorted by Quantity (High to Low)
5. Exit
Enter your choice (1-5): 5
4. Show Products Sorted by Quantity (High to Low)
5. Exit
Enter your choice (1-5): 5
Exiting.
```

Observation:

For the inventory system scenario, AI suggested using Binary Search for quick product lookups and efficient sorting algorithms (Merge Sort/Quick Sort) for arranging products by price or quantity. It justified these choices based on dataset size and performance requirements. The AI also provided a Python implementation that allowed searching by ID/name and sorting using options. Through this, I observed how AI applies theoretical algorithms to solve real-world problems, while also justifying the decisions with clear reasoning.